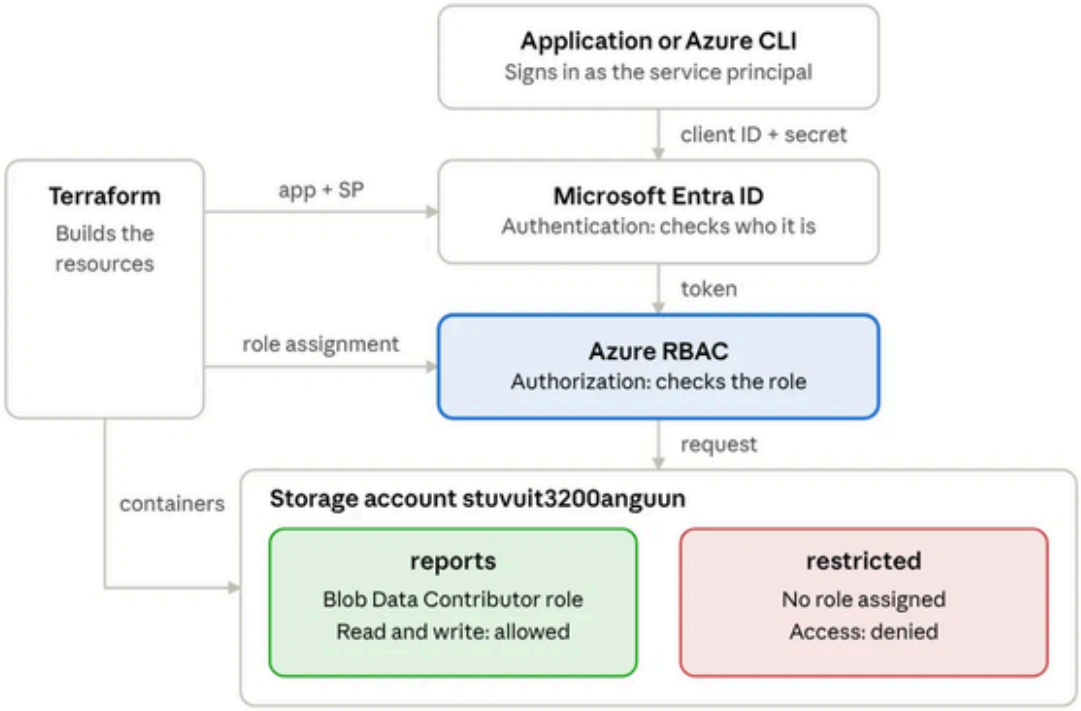
Architecture



The app signs in with Entra ID, which checks who it is and hands back a token. Azure RBAC then checks the role on each request. The storage account and containers are deployed.

Infrastructure

I built this with Terraform. It creates a resource group, a storage account (HTTPS only, TLS 1.2, no public blob access) and two private containers, reports and restricted. Stage 1 terraform apply worked and added 4 resources.
The Entra app, service principal and role assignment are written in service_principal.tf. They are switched off by default with create_service_principal and assign_blob_role, so I could deploy the storage first and add the identity after.

Identity and Authorization

The app registration is the definition of the app in Entra ID. The service principal is the identity made from it. I picked Storage Blob Data Contributor because the app needs to upload, list and download. Reader is too little, Owner is too much, and Owner or Contributor on the account are management roles that do not even give blob data access

Security

The app gets its own identity, one data role and only one container, with no keys and no management role. The storage account blocks public access and requires HTTPS. terraform.tfvars and state files are in .gitignore and are not in the repo.

Reflection

I also learned why least privilege matters. I picked Storage Blob Data Contributor because the app only needs to upload, list and download, and I scoped it to the reports container so the restricted container stays off limits. Owner and Contributor are management roles, so they would have given far more than the app needs. If I did this again for a real app, I would use a managed identity so there is no client secret at all, store the Terraform state in a lockeddown remote backend, and turn on storage logging so access to the containers can be audited.